

第3章 动态规划

3.7 图像压缩

在计算机中，常用像素点的灰度值序列 $\{p_1, p_2, \dots, p_n\}$ 表示图像。其中整数 $p_i, 1 \leq i \leq n$ ，表示像素点 i 的灰度值。通常灰度值的范围是0~255。因此最多需要8位表示一个像素。



■ 一张分辨率为640 x 480的图片，那它的像素就达到了307200，也就是人们常说的30万像素

图像压缩

图象的变位压缩存储格式将所给的像素点序列 $\{p_1, p_2, \dots, p_n\}$, $0 \leq p_i \leq 255$ 分割成 m 个连续段 $S_1, S_2, \dots, S_m$ 。

第 i 个像素段 S_i 中 ($1 \leq i \leq m$), 有 $l[i]$ 个像素, 且该段中每个像素都只用 $b[i]$ 位表示。

分段的过程就是要找出断点, 让一段里面的像素的最大灰度值比较小, 那么这一段像素(本来需要8位)就可以用较少的位(比如7位)来表示, 从而减少存储空间。

设 $t[i] = \sum_{k=1}^{i-1} l[k]$ 则第 i 个像素段 S_i 为

$$S_i = \{p_{t[i]+1}, \dots, p_{t[i]+l[i]}\}$$

比如某个片段为：

$p_i=10$, $p_{i+1}=15$, $p_{i+2}=100$, $p_{i+3}=55$, $p_{i+4}=200$, $p_{i+5}=255$

可以分成 p_i-p_{i+3} 和 $p_{i+4}-p_{i+5}$ 两段，而第一段的每个像素只需要7个比特位就可以表示。

图像压缩

本题中, $0 \leq p_i \leq 255$, 因此 $b[i] \leq 8$, 即需要用3位表示 $b[i]$, 如果限制每段不超过255个像素, 即 $1 \leq l[i] \leq 255$, 则需要用8位表示 $l[i]$ 。因此, 第 i 个像素段所需的存储空间为 $l[i]*b[i]+8+3= l[i]*b[i]+11$ 位。

按此格式存储像素序列 $\{p_1, p_2, \dots, p_n\}$, 需要 $\sum_{i=1}^m l[i]*b[i]+11m$ 位的存储空间。

图象压缩问题要求确定像素序列 $\{p_1, p_2, \dots, p_n\}$ 的最优分段, 使得依此分段所需的存储空间最少。每个分段的像素不超过256个。

图像压缩

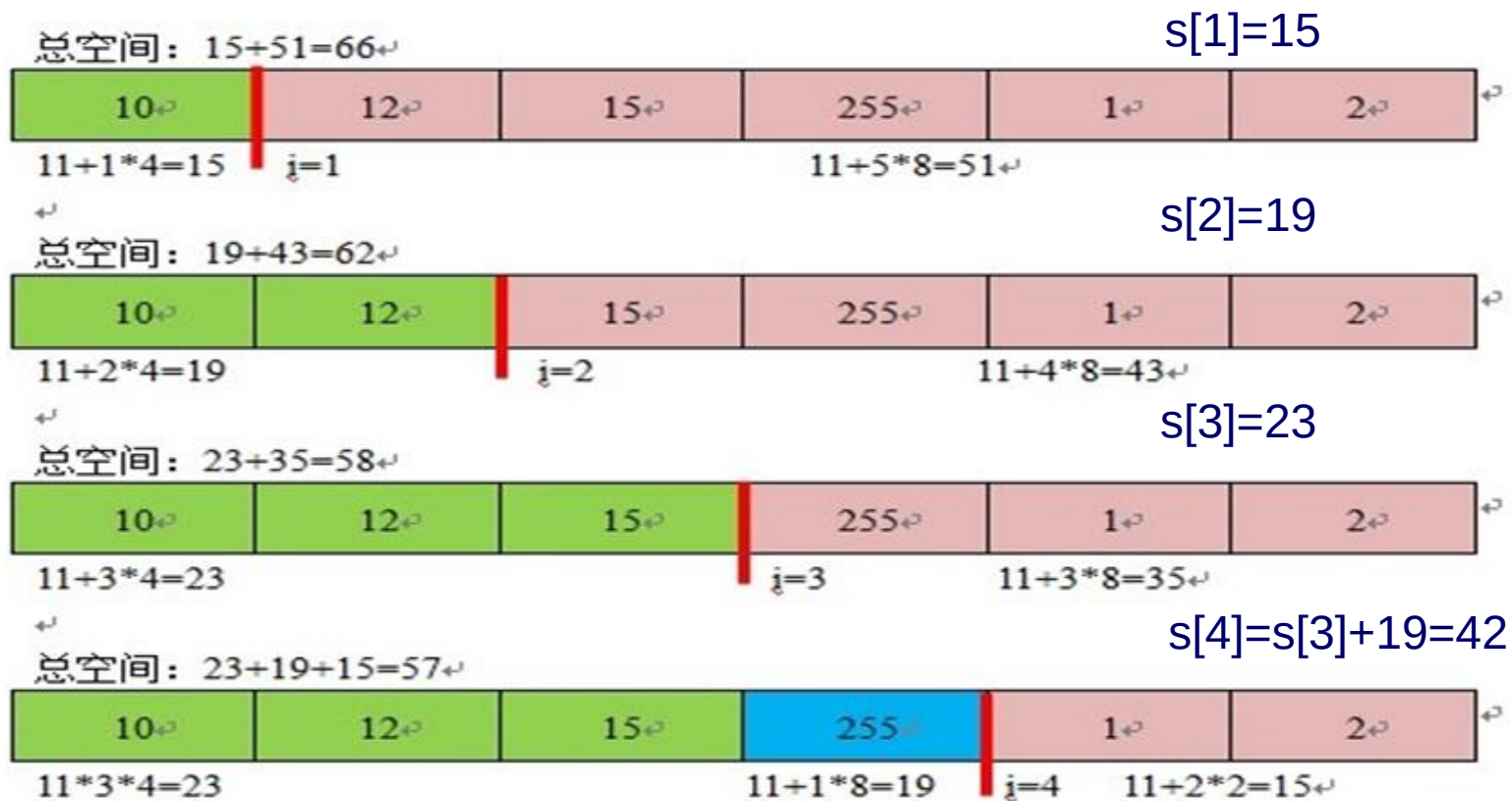
设 $l[i]$, $b[i]$, $1 \leq i \leq m$, 是 $\{p_1, p_2, \dots, p_n\}$ 的最优分段。
 $l[1]$, $b[1]$ 是 $\{p_1, \dots, p_{l[1]}\}$ 的最优分段, 且 $l[i]$, $b[i]$, $2 \leq i \leq m$ 是 $\{p_{l[1]+1}, \dots, p_n\}$ 的最优分段。即图象压缩问题满足最优子结构性质。

设 $s[i]$, $1 \leq i \leq n$, 是像素序列 $\{p_1, \dots, p_i\}$ 的最优分段所需的存储位数, 则 $s[i]$ 为前 $i-k$ 个的存储位数 (已算过, 是 $s[i-k]$) 加上后 k 个的存储位数 (需再计算)。由最优子结构性质易知:

$$s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * b_{\max(i-k+1, i)}\} + 11$$

其中 $b_{\max(i, j)} = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil$ 为 p_i 到 p_j 中, 最大的值需要的比特位数。

例：6个像素，值分别为10,12,15,255,1,2



算法复杂度分析：

由于算法compress中对k的循环次数不超过256，故对每一个确定的i，可在时间O(1)内完成的计算。因此整个算法所需的计算时间为O(n)。

$$s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * b \max(i-k+1, i)\} + 1$$

3.9 流水作业调度(课堂不讲)

n 个作业 $\{1, 2, \dots, n\}$ 要在由2台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工，然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i 和 b_i 。流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 M_1 上开始加工，到最后一个作业在机器 M_2 上加工完成所需的时间最少。

分析：

- 直观上，一个最优调度应使机器 M_1 没有空闲时间，且机器 M_2 的空闲时间最少。在一般情况下，机器 M_2 上会有机器空闲和作业积压2种情况。
- 设全部作业的集合为 $N=\{1, 2, \dots, n\}$ 。 $S \subseteq N$ 是 N 的作业子集。在一般情况下，机器 M_1 开始加工 S 中作业时，机器 M_2 还在加工其它作业，要等时间 t 后才可利用。将这种情况下完成 S 中作业所需的最短时间记为 $T(S, t)$ 。流水作业调度问题的最优值为 $T(N, 0)$ 。

流水作业调度

设 π 是所给 n 个流水作业的一个最优调度，它所需的加工时间为 $a_{\pi(1)} + T'$ 。其中 T' 是在机器 M_2 的等待时间为 $b_{\pi(1)}$ 时，安排作业 $\pi(2), \dots, \pi(n)$ 所需的时间。

记 $S = N - \{\pi(1)\}$ ，则有 $T' = T(S, b_{\pi(1)})$ 。

证明：事实上，由 T 的定义知 $T' \geq T(S, b_{\pi(1)})$ 。若 $T' > T(S, b_{\pi(1)})$ ，设 π' 是作业集 S 在机器 M_2 的等待时间为 $b_{\pi(1)}$ 情况下的一个最优调度。则 $\pi(1), \pi'(2), \dots, \pi'(n)$ 是 N 的一个调度，且该调度所需的时间为 $a_{\pi(1)} + T(S, b_{\pi(1)}) < a_{\pi(1)} + T'$ 。这与 π 是 N 的最优调度矛盾。故 $T' \leq T(S, b_{\pi(1)})$ 。从而 $T' = T(S, b_{\pi(1)})$ 。这就证明了流水作业调度问题具有最优子结构的性质。

由流水作业调度问题的最优子结构性性质可知，

$$T(N, 0) = \min_{1 \leq i \leq n} \{a_i + T(N - \{i\}, b_i)\}$$

$$T(S, t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

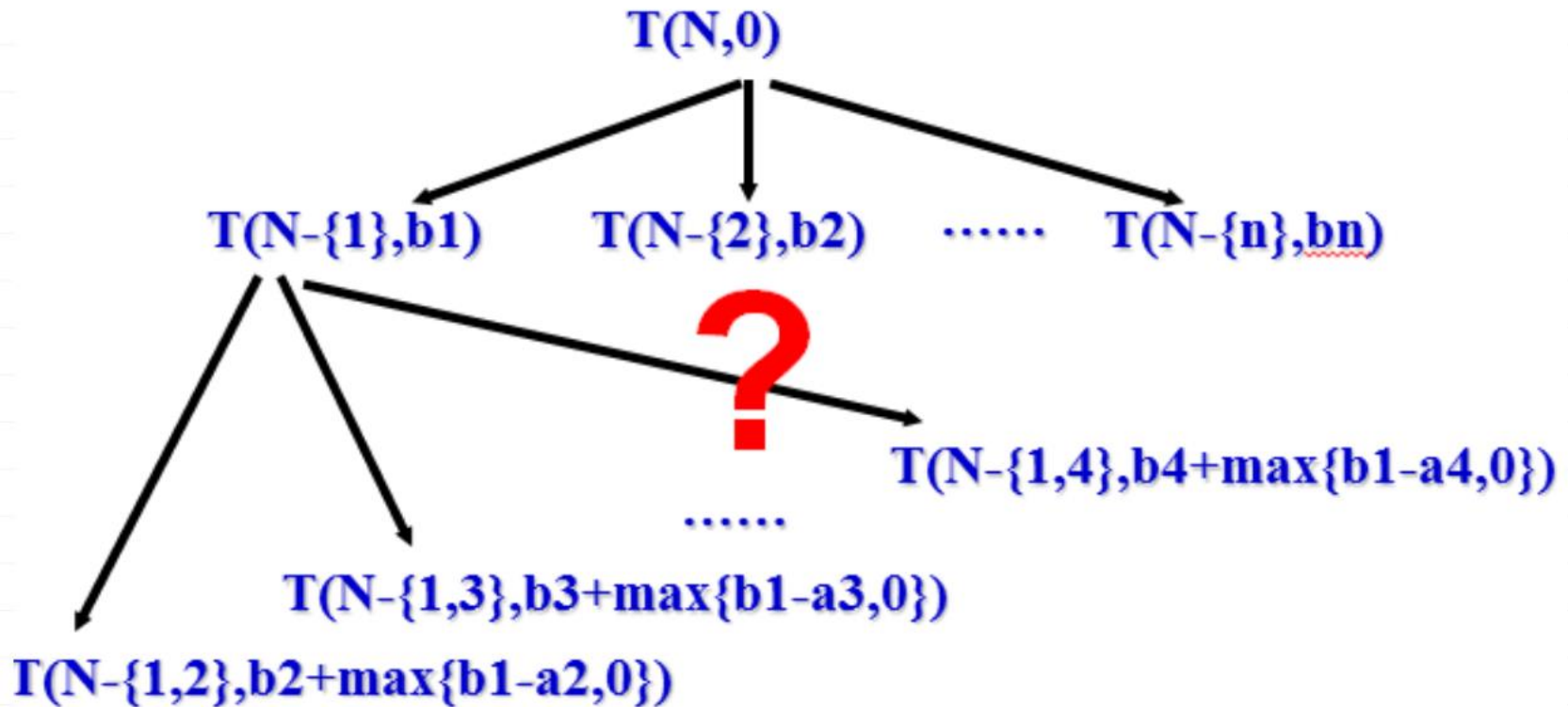
选定作业 i 为 S 中第一个加工作业之后，在机器 M_2 上开始对 $S-\{i\}$ 中的作业进行加工之前，所需要的等待时间为 $b_i + \max\{t-a_i, 0\}$ 。

原因：若 M_2 在开始加工 S 中的作业之前需等待 t 个时间单位，且 $t > a_i$ ，则作业 i 在 M_1 上加工完毕（需时 a_i ）之后，还要再等 $t-a_i$ 个时间单位才能开始在 M_2 上加工；若 $t \leq a_i$ ，则作业 i 在 M_1 上加工完毕之后，立即可以在 M_2 上加工，等待时间为0。故 M_2 在开始对 $S-\{i\}$ 中的作业进行加工之前，所需要的等待时间为 $t' = b_i + \max\{t-a_i, 0\}$ 。（ b_i 是作业 i 在 M_2 上加工所需的时间）。所以，假定 a_i 为已知的使得 $T(S, t)$ 值最小的第一个执行的作业，可以得到

$$T(S, t) = a_i + T(S-\{i\}, b_i + \max\{t-a_i, 0\})$$

流水作业调度问题

■ 子问题重叠性质



虽然满足最优子结构性质，也在一定程度满足子问题重叠性质。N的每个非空子集都计算一次，共 2^n-1 次，指数级的。

为了解决这个问题引入Johnson不等式

Johnson不等式

对递归式的深入分析表明，算法可进一步得到简化。

设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得：

$$T(S,t)=a_i+T(S-\{i\},b_i+\max\{t-a_i,0\})=a_i+a_j+T(S-\{i,j\},t_{ij})$$

$$\begin{aligned}\text{其中, } t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\ &= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\ &= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\ &= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}\end{aligned}$$

如果作业 i 和 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则称作业 i 和 j 满足**Johnson不等式**。

流水作业调度的Johnson法则

$$t_{ij} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}$$

交换作业i和作业j的加工顺序，得到作业集S的另一调度，它所需的加工时间为 $T'(S, t) = a_i + a_j + T(S - \{i, j\}, t_{ji})$

$$\text{其中, } t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$$

当作业i和j满足Johnson不等式时，有

$$\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$$

$$a_i + a_j + \max\{-b_i, -a_j\} \leq a_i + a_j + \max\{-b_j, -a_i\}$$

$$\max\{a_i + a_j - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$$

$$\max\{t, a_i + a_j - b_i, a_i\} \leq \max\{t, a_i + a_j - b_j, a_j\}$$

由此可见当作业i和作业j满足Johnson不等式时，交换它们的加工顺序后，会增加加工时间。对于流水作业调度问题，必存在最优调度 π ，使得任意i，作业 $\pi(i)$ 和 $\pi(i+1)$ 满足Johnson不等式。进一步还可以证明，调度满足Johnson法则当且仅当对任意 $i < j$ 有

$$\min\{b_{\pi(i)}, a_{\pi(j)}\} \geq \min\{b_{\pi(j)}, a_{\pi(i)}\}$$

由此可知，所有满足Johnson法则的调度均为最优调度。

算法描述

流水作业调度问题的Johnson算法

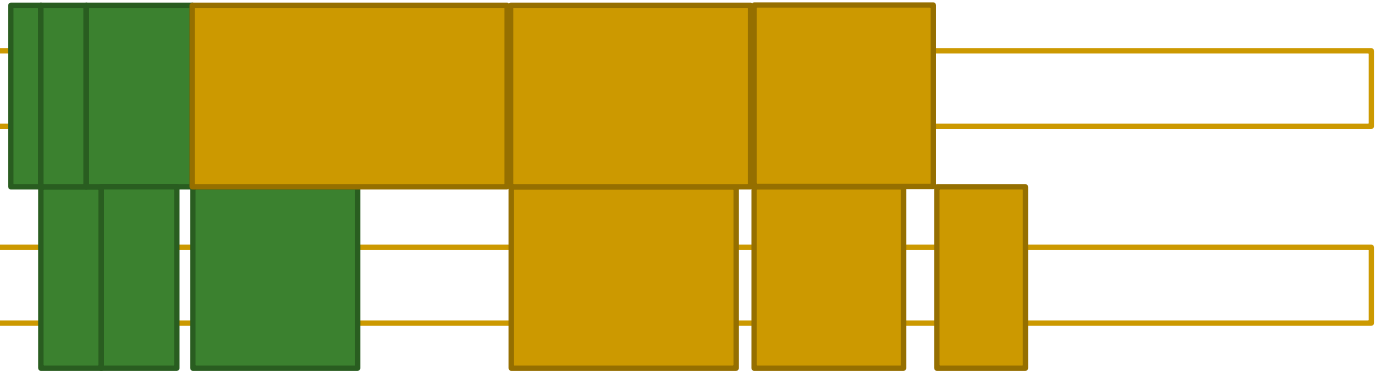
- (1) 令 $N_1 = \{i \mid a_i < b_i\}$, $N_2 = \{i \mid a_i \geq b_i\}$;
- (2) 将 N_1 中作业依 a_i 的非减序排序; 将 N_2 中作业依 b_i 的非增序排序;
- (3) N_1 中作业接 N_2 中作业构成满足Johnson法则的最优调度。

算法复杂度分析:

算法的主要计算时间花在对作业集的排序。因此, 在最坏情况下算法所需的计算时间为 $O(n \log n)$ 。所需的空间为 $O(n)$ 。

M1

M2

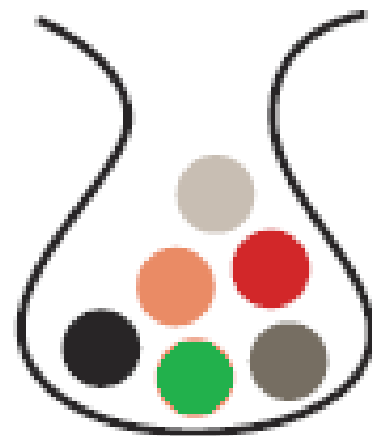


3.10 0-1背包问题

给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 C 。问应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

0-1背包问题是一个特殊的整数规划问题。

$$\begin{aligned} & \max \sum_{i=1}^n v_i x_i \\ & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$



0-1背包问题

设所给0-1背包问题的子问题

$$\begin{cases} \max \sum_{k=i}^n v_k x_k \\ \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\}, i \leq k \leq n \end{cases}$$

■若背包剩余容量大于第i个物品重量，则分别考虑装与不装两种情况的重量，取最大值

的最优值为 $m(i, j)$ ，即 $m(i, j)$ 是背包容量为 j ，可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质，可以建立计算 $m(i, j)$ 的递归式如下。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

■若背包剩余容量小于第i个物品重量，则不考虑这个物品

```

void Knapsack(int v[],int w[],int c,int n,int m[][10])
{
    int jMax = min(w[n]-1,c);//背包剩余容量上限 范围[0~w[n]-1]
    for(int j=0; j<=jMax;j++)
    {
        m[n][j]=0;
    }
    for(int j=w[n]; j<=c; j++)//限制范围[w[n]~c]
    {
        m[n][j] = v[n];
    }

    for(int i=n-1; i>1; i--)
    {
        jMax = min(w[i]-1,c);
        for(int j=0; j<=jMax; j++)//背包不同剩余容量j<=jMax<c
        {
            m[i][j] = m[i+1][j];//没产生任何效益
        }
        for(int j=w[i]; j<=c; j++) //背包不同剩余容量j-wi > c
        {
            m[i][j] = max(m[i+1][j],m[i+1][j-w[i]]+v[i]);//效益值增长vi
        }
    }
    m[1][c] = m[2][c];
    if(c>=w[1])
    {
        m[1][c] = max(m[1][c],m[2][c-w[1]]+v[1]);
    }
}

```

//x[]数组存储对应物品0-1向量,0不装入背包, 1表示装入背包

```

void Traceback(int m[][10],int w[],int c,int n,int x[])
{
    for(int i=1; i<n; i++)
    {
        if(m[i][c] == m[i+1][c])
        {
            x[i]=0;
        }
        else
        {
            x[i]=1;
            c-=w[i];
        }
    }
    x[n]=(m[n][c])?1:0;
}

```

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$n=4 \quad c=8$

$w[] = \{1, 4, 2, 3\} \quad v[] = \{2, 1, 4, 3\}$

$j \backslash i$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$	$j=8$
$i=4$	0	0	3	3	3	3	3	3
$i=3$	0	4	4	4	7	7	7	7
$i=2$	0	4	4	4	7	7	7	7
$i=1$								9

绿色框为方法 Traceback 的回溯过程 $x[] = \{1, 0, 1, 1\}$

算法复杂度分析：

从Knapsack容易看出，算法需要 $O(nc)$ 计算时间。当背包容量 c 很大时，算法需要的计算时间较多。例如，当 $c > 2^n$ 时，算法需要 $\Omega(n2^n)$ 计算时间。

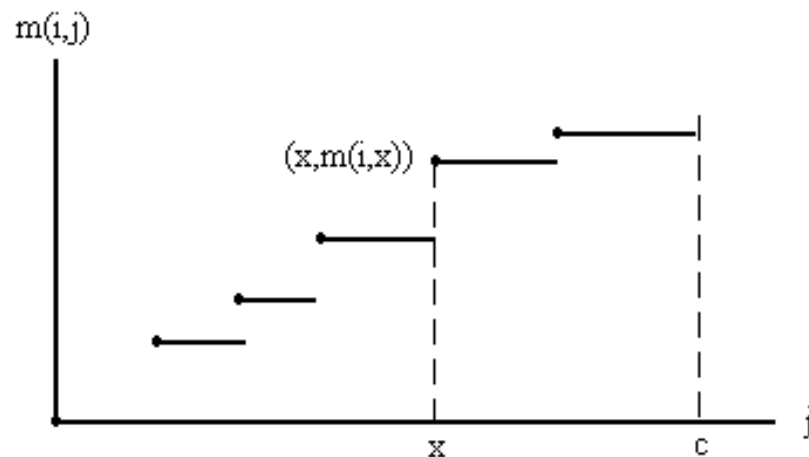
算法改进

$$n = 4 \quad c = 8$$

$$w[] = \{1, 4, 2, 3\} \quad v[] = \{2, 1, 4, 3\}$$

$j \backslash i$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$	$j=8$
$i=4$	0	0	3	3	3	3	3	3
$i=3$	0	4	4	4	7	7	7	7
$i=2$	0	4	4	4	7	7	7	7
$i=1$								9

由 $m(i, j)$ 的递归式容易证明，在一般情况下，对每一个确定的 $i (1 \leq i \leq n)$ ，函数 $m(i, j)$ 是关于变量 j 的阶梯状单调不减函数。跳跃点是这一类函数的描述特征。在一般情况下，函数 $m(i, j)$ 由其全部跳跃点唯一确定。如图所示。



对每一个确定的 $i (1 \leq i \leq n)$ ，用一个表 $p[i]$ 存储函数 $m(i, j)$ 的全部跳跃点。表 $p[i]$ 可根据计算 $m(i, j)$ 的递归式来递归地由表 $p[i+1]$ 计算，初始时 $p[n+1] = \{(0, 0)\}$ 。

算法改进

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

• 函数 $m(i, j)$ 是由函数 $m(i+1, j)$ 与函数 $m(i+1, j-w_i) + v_i$ 作 $\max$ 运算得到的。因此，函数 $m(i, j)$ 的全部跳跃点包含于函数 $m(i+1, j)$ 的跳跃点集 $p[i+1]$ 与函数 $m(i+1, j-w_i) + v_i$ 的跳跃点集 $q[i+1]$ 的并集中。易知， $(s, t) \in q[i+1]$ 当且仅当 $w_i \leq s \leq c$ 且 $(s-w_i, t-v_i) \in p[i+1]$ 。因此，容易由 $p[i+1]$ 确定跳跃点集 $q[i+1]$ 如下

$$q[i+1] = p[i+1] \oplus (w_i, v_i) = \{(j+w_i, m(i, j) + v_i) \mid (j, m(i, j)) \in p[i+1]\}$$

• 另一方面，设 (a, b) 和 (c, d) 是 $p[i+1] \cup q[i+1]$ 中的2个跳跃点，则当 $c \geq a$ 且 $d < b$ 时， (c, d) 受控于 (a, b) ，从而 (c, d) 不是 $p[i]$ 中的跳跃点。除受控跳跃点外， $p[i+1] \cup q[i+1]$ 中的其它跳跃点均为 $p[i]$ 中的跳跃点。

• 由此可见，在递归地由表 $p[i+1]$ 计算表 $p[i]$ 时，可先由 $p[i+1]$ 计算出 $q[i+1]$ ，然后合并表 $p[i+1]$ 和表 $q[i+1]$ ，并清除其中的受控跳跃点得到表 $p[i]$ 。

一个例子

$n=5, c=10, w=\{2, 2, 6, 5, 4\}, v=\{6, 3, 5, 4, 6\}$ 。

初始时 $p[6]=\{(0,0)\}$, $(w_5,v_5)=(4,6)$ 。因此,

$q[6]=p[6]\oplus(w_5,v_5)=\{(4,6)\}$ 。

$p[5]=\{(0,0),(4,6)\}$ 。

$q[5]=p[5]\oplus(w_4,v_4)=\{(5,4),(9,10)\}$ 。从跳跃点集 $p[5]$ 与 $q[5]$ 的并集

$p[5]\cup q[5]=\{(0,0),(4,6),(5,4),(9,10)\}$ 中看到跳跃点 $(5,4)$ 受控于跳跃点 $(4,6)$ 。将受控跳跃点 $(5,4)$ 清除后, 得到

$p[4]=\{(0,0),(4,6),(9,10)\}$

$q[4]=p[4]\oplus(6, 5)=\{(6, 5), (10, 11)\}$

$p[3]=\{(0, 0), (4, 6), (9, 10), (10, 11)\}$

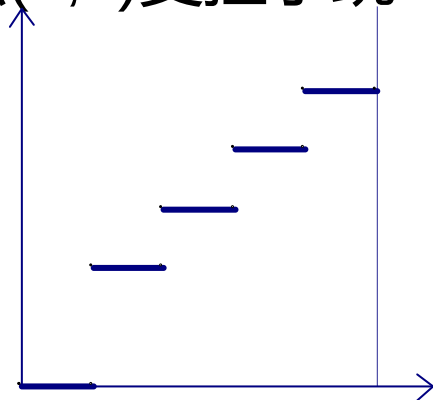
$q[3]=p[3]\oplus(2, 3)=\{(2, 3), (6, 9)\}$

$p[2]=\{(0, 0), (2, 3), (4, 6), (6, 9), (9, 10), (10, 11)\}$

$q[2]=p[2]\oplus(2, 6)=\{(2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]=\{(0, 0), (2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]$ 的最后的那个跳跃点 $(8,15)$ 给出所求的最优值为 $m(1,c)=15$ 。



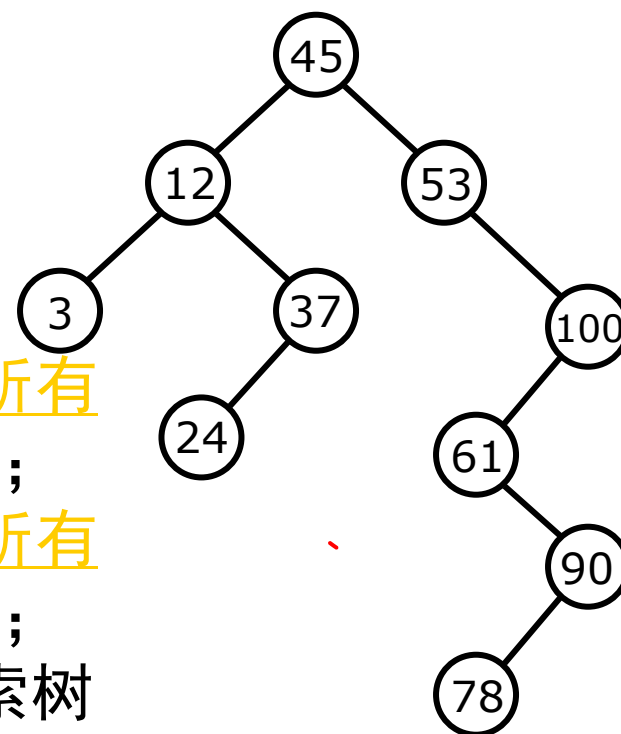
算法复杂度分析

上述算法的主要计算量在于计算跳跃点集 $p[i](1 \leq i \leq n)$ 。由于 $q[i+1] = p[i+1] \oplus (w_i, v_i)$ ，故计算 $q[i+1]$ 需要 $O(|p[i+1]|)$ 计算时间。合并 $p[i+1]$ 和 $q[i+1]$ 并清除受控跳跃点也需要 $O(|p[i+1]|)$ 计算时间。从跳跃点集 $p[i]$ 的定义可以看出， $p[i]$ 中的跳跃点相应于 $x_1, \dots, x_n$ 的 0/1 赋值。因此， $p[i]$ 中跳跃点个数不超过 2^{n-i+1} 。由此可见，算法计算跳跃点集 $p[i]$ 所花费的计算时间为 $O\left(\sum_{i=2}^n |p[i+1]|\right) = O\left(\sum_{i=2}^n 2^{n-i}\right) = O(2^n)$ 。从而，改进后算法的计算时间复杂性为 $O(2^n)$ 。当所给物品的重量 $w_i(1 \leq i \leq n)$ 是整数时， $|p[i]| \leq c+1$ ， $(1 \leq i \leq n)$ 。在这种情况下，改进后算法的计算时间复杂性为 $O(\min\{nc, 2^n\})$ 。

3.11 最优二叉搜索树

■ 二叉搜索树

- (1) 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值；
- (2) 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值；
- (3) 它的左、右子树也分别为二叉搜索树



在随机的情况下，二叉查找树的平均查找长度和 $\log n$ 是等数量级的

二叉查找树的期望耗费

关键字 $k_1, \dots, k_n$ 和伪关键字 $d_0, \dots, d_n$ 分别有概率 p_i, q_i

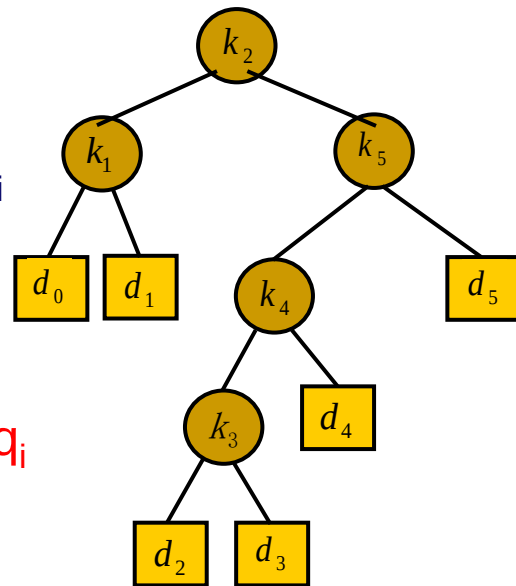
- 查找成功与不成功的概率

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

找到元素 k_i 的概率为 p_i

在 (k_i, k_{i+1}) 区间的概率为 q_i

- 二叉查找树的期望耗费

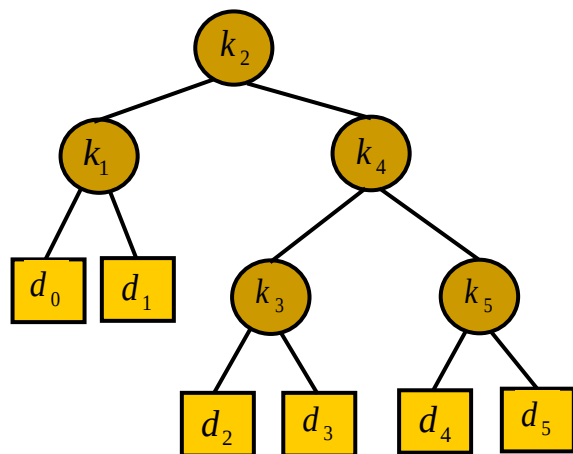


假设一次搜索的实际代价为检查的节点的个数，这里假设根节点深度为0，在 T 中计算一次搜索的期望代价等式为：

$$E[T \text{ 中的搜索代价}] = \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i$$

- 最优二叉搜索树 问题是对有序集 S 及其概率分布 $(q_0, p_1, q_1, \dots, p_n, q_n)$ ，在所有表示有序集 S 的二叉搜索树中找出一棵有最小平均路长的二叉搜索树。

二叉查找树的期望耗费示例



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.10
d_1	2	0.10	0.20
d_2	3	0.05	0.15
d_3	3	0.05	0.15
d_4	3	0.05	0.15
d_5	3	0.10	0.30
Total			2.40

最优二叉搜索树

在搜索过程中，每进行一次比较，就进入下面一层，

- 对于成功的搜索，比较的次数就是所在的层数加1。
- 对于不成功的搜索，被搜索的关键字属于哪个外部结点代表的可能关键字集合，比较次数就等于此外部结点的层数。

最优二叉搜索树

- 找到元素 $x = x_i$ 的概率为 b_i ; 确定 $x \in (x_i, x_{i+1})$ 的概率为 a_i 。其中约定 $x_0 = -\infty$, $x_{n+1} = +\infty$, 有

$$a_i \geq 0, 0 \leq i \leq n; b_j \geq 0, 1 \leq j \leq n; \sum_{i=0}^n a_i + \sum_{j=1}^n b_j = 1$$

称集合 $\{a_0, b_1, a_1, \dots, b_n, a_n\}$ 为有序集 S 的存取概率分布。

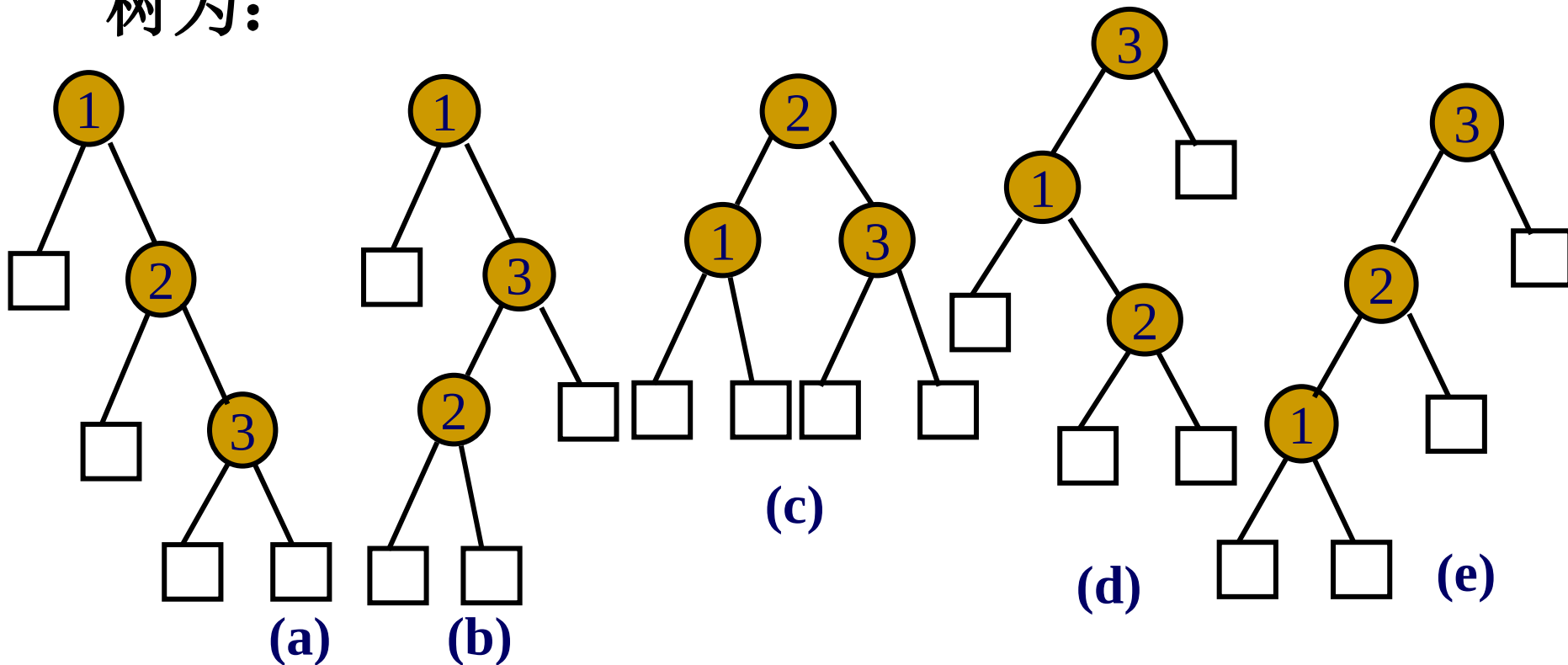
2 最优二叉搜索树

■ 在一个表示S的二叉树T中，设存储元素 x_i 的结点深度为 c_i ；叶结点 (x_j, x_{j+1}) 的结点深度为 d_j 。

$$p = \sum_{i=1}^n b_i (1 + c_i) + \sum_{j=0}^n a_j d_j$$

表示在二叉搜索树T中作一次搜索所需的平均比较次数。 p 又称为二叉搜索树T的平均路长，在一般情况下，不同的二叉搜索树的平均路长是不同的。

例 标识符集{1, 2, 3}={do, if, stop}可能的二叉搜索树为:



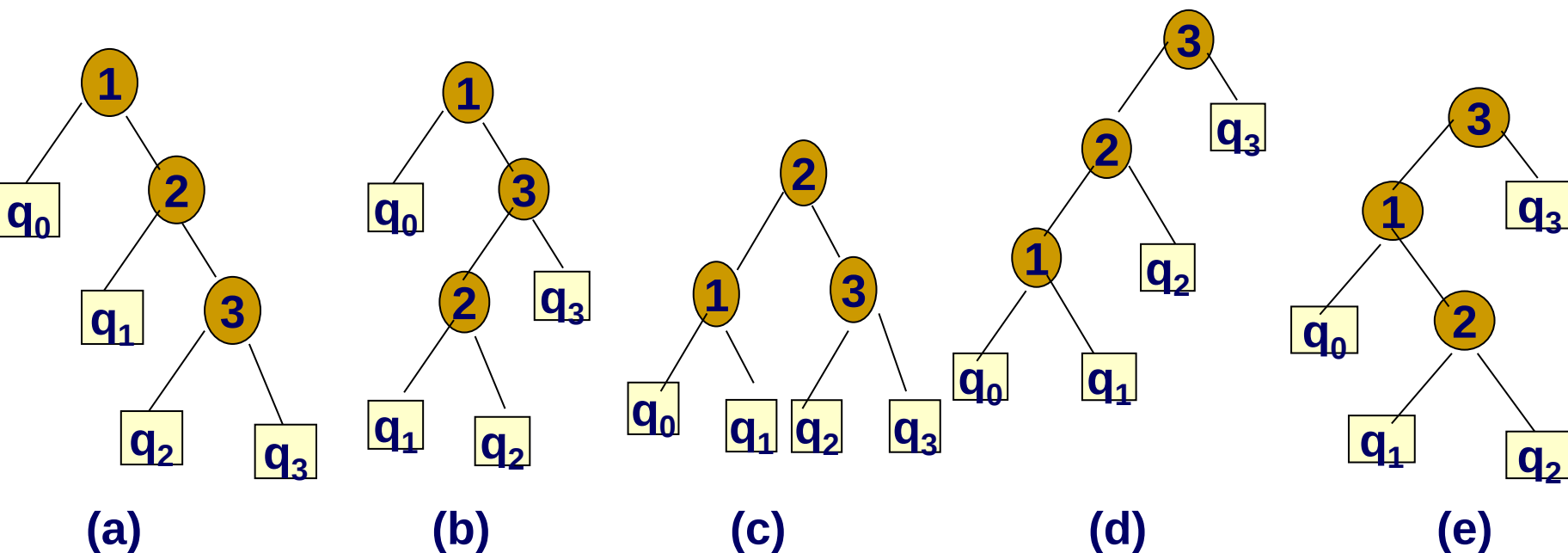
设每个内、外结点检索的概率相同: $p_i=q_i=1/7$,
求每棵树的平均比较次数 (成本)。

若 $P_1=0.5$, $P_2=0.1$, $P_3=0.05$,

$q_0=0.15$, $q_1=0.1$, $q_2=0.05$, $q_3=0.05$, 求每棵树的平均比较次数 (成本)。

2 最优二叉搜索树

例: $P_1=0.5$, $P_2=0.1$, $P_3=0.05$, $q_0=0.15$, $q_1=0.1$, $q_2=0.05$, $q_3=0.05$



考虑平均搜索次数，也叫做平均路长

$$\begin{aligned} P_a(n) &= 1 \times p_1 + 2 \times p_2 + 3 \times p_3 + 1 \times q_0 + 2 \times q_1 + 3 \times (q_2 + q_3) \\ &= 1 \times 0.5 + 2 \times 0.1 + 3 \times 0.05 + 1 \times 0.05 + 2 \times 0.1 + 3 \times (0.05 + 0.05) \\ &= 1.5 \end{aligned}$$

2 最优二叉搜索树

分析

对于图的内结点而言，第0层需要比较操作次数为1，第1层需要比较2次，第2层需要3次

$$\begin{aligned}P_b(n) &= 1 \times p_1 + 2 \times p_3 + 3 \times p_2 + 1 \times q_0 + 3 \times (q_2 + q_3) \\&= 1 \times 0.5 + 2 \times 0.05 + 3 \times 0.1 + 1 \times 0.15 + 3 \times (0.05 + 0.05) \\&= 1.6\end{aligned}$$

$$\begin{aligned}P_c(n) &= 1 \times p_2 + 2 \times (p_1 + p_3) + 2 \times (q_0 + q_1 + q_2 + q_3) \\&= 1 \times 0.1 + 2 \times (0.5 + 0.05) + 2 \times (0.15 + 0.1 + 0.05 + 0.05) \\&= 1.9\end{aligned}$$

$$\begin{aligned}P_d(n) &= 1 \times p_3 + 2 \times p_1 + 3 \times p_2 + 1 \times q_3 + 2 \times q_0 + 3 \times (q_1 + q_2) \\&= 1 \times 0.05 + 2 \times 0.5 + 3 \times 0.1 + 1 \times 0.05 + 2 \times 0.15 + 3 \times (0.1 + 0.05) \\&= 2.15\end{aligned}$$

$$\begin{aligned}P_e(n) &= 1 \times p_3 + 2 \times p_1 + 3 \times p_2 + 1 \times q_3 + 2 \times q_0 + 3 \times (q_1 + q_2) \\&= 1 \times 0.05 + 2 \times 0.5 + 3 \times 0.1 + 1 \times 0.05 + 2 \times 0.15 + 3 \times (0.1 + 0.05) \\&= 2.15\end{aligned}$$

3 最优二叉搜索树问题描述

- 对于有序集 $\mathbf{S}$ 及其存取概率分布

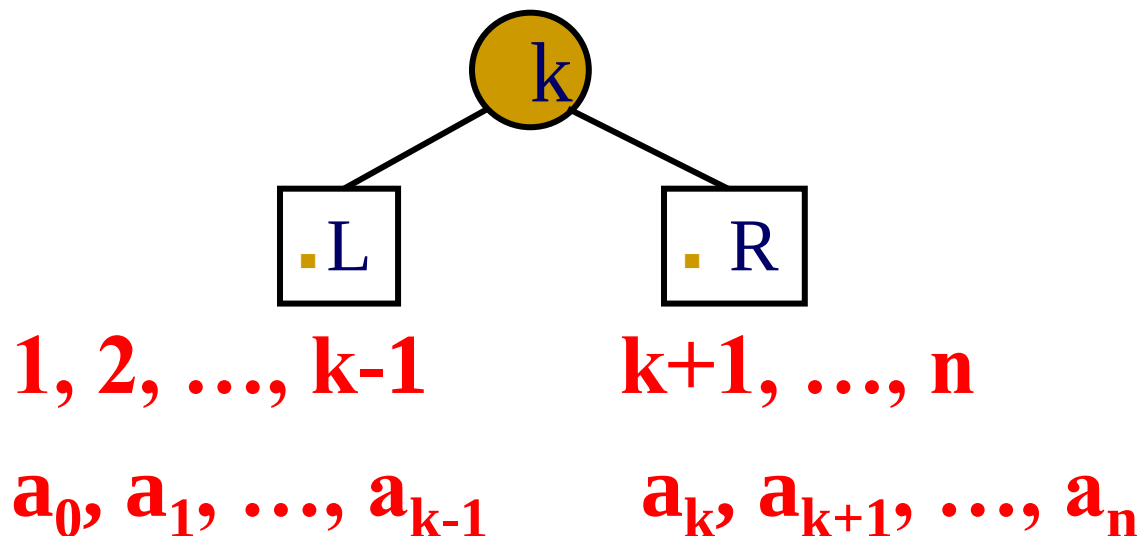
$(a_0, b_1, a_1, \dots, b_n, a_n)$ ，在所有表示有序集 $\mathbf{S}$ 的二叉搜索树中找出一棵具有最小平均路长的二叉搜索树。

结点在二叉搜索树中的层次越深，需要比较的次数就越多，因此要构造一棵最小平均路长的二叉搜索树，一般尽量把搜索概率较高的结点放在较高的层次。

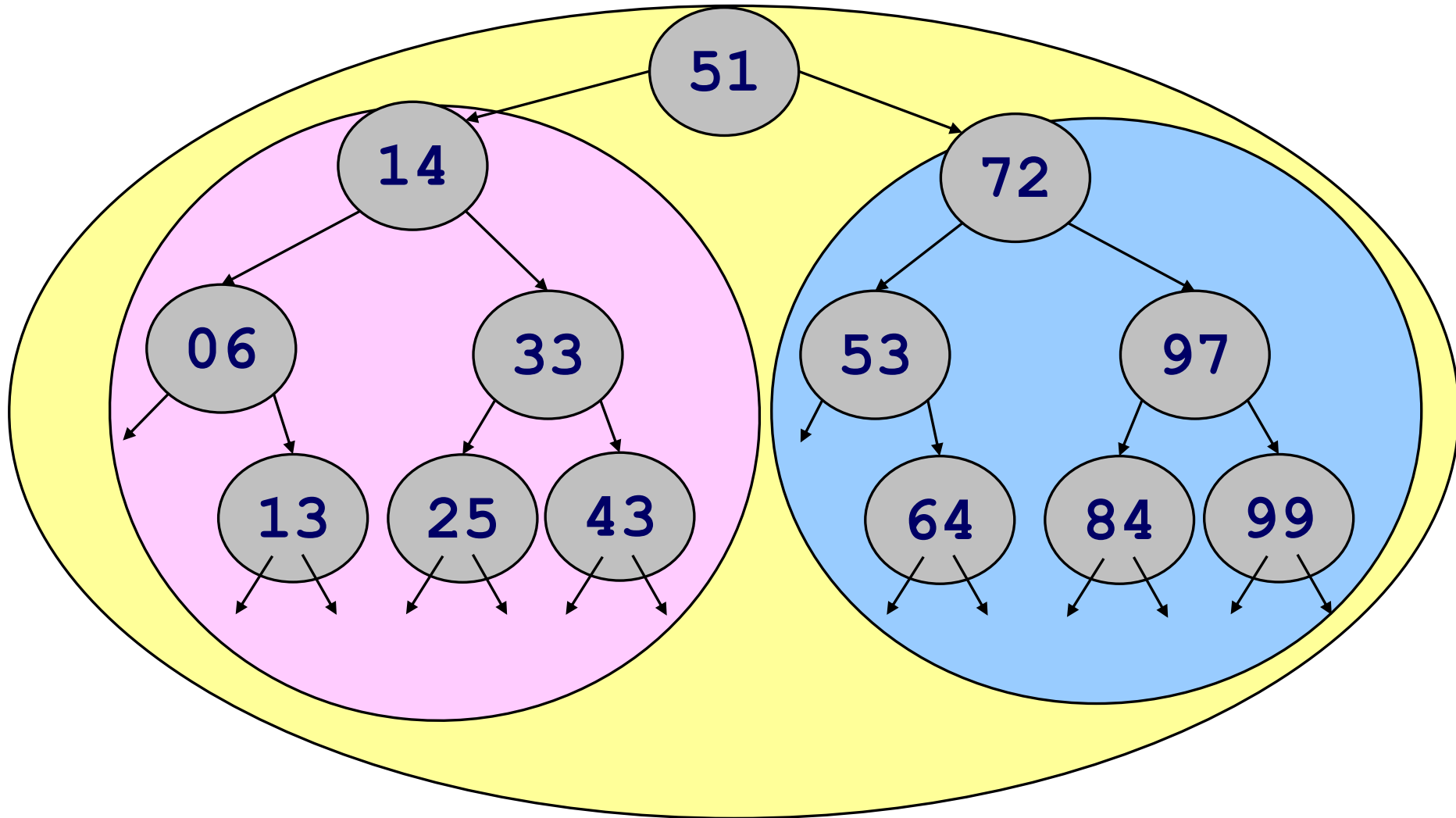
4、最优子结构性质

假设选择 k 为树根，则 $1, 2, \dots, k-1$ 和 $a_0, a_1, \dots, a_{k-1}$ 都将位于左子树 L 上，

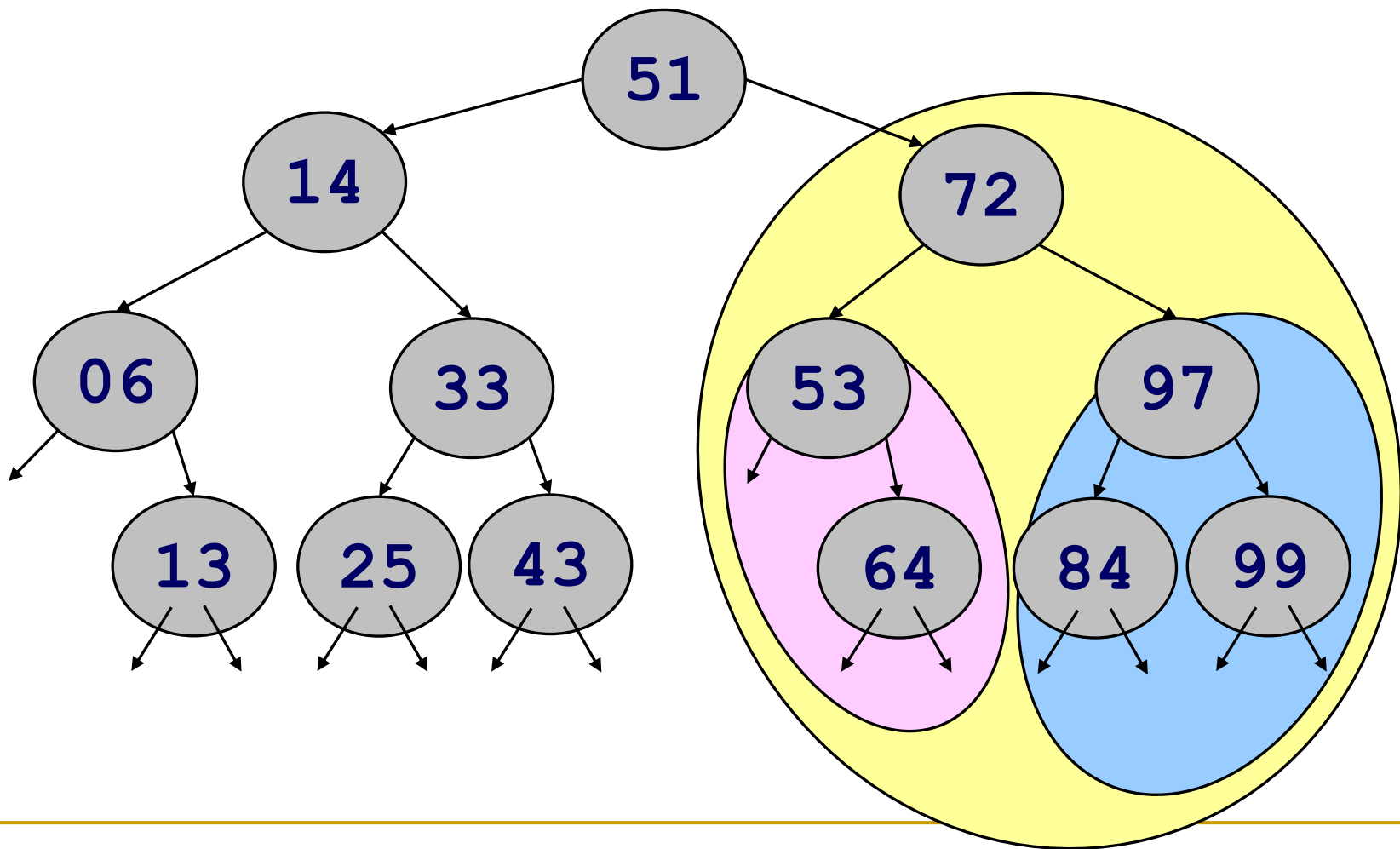
其余结点 ($k+1, \dots, n$ 和 $a_k, a_{k+1}, \dots, a_n$) 位于右子树 R 上。



4 最优子结构性质



4 最优子结构性质



4 最优子结构性质

设 $\text{COST}(L)$ 和 $\text{COST}(R)$ 分别是二叉搜索树 T 的左子树和右子树的成本。

则检索树 T 的成本是：

$$P(k) + \text{COST}(L) + \text{COST}(R) + \dots$$

若 T 是最优的，则上式及 $\text{COST}(L)$ 和 $\text{COST}(R)$ 必定都取最小值。

4 最优子结构性性质

最优子结构性性质证明

■ 二叉搜索树 T 的一棵含有顶点 $x_i, \dots, x_j$ 和叶顶点 $(x_{i-1}, x_i), \dots, (x_j, x_{j+1})$ 的子树可以看作是有序集 $\{x_i, \dots, x_j\}$ 关于全集为 $\{x_{i-1}, x_{j+1}\}$ 的一棵二叉搜索树 (T 自身可以看作是有序集)。

根据 S 的存取分布概率，在子树的顶点处被搜索到的概率是：

$$w_{ij} = \sum_{i-1 \leq k \leq j} a_k + \sum_{i \leq k \leq j} b_k$$

4 最优子结构性质

- $\{x_i, \dots, x_j\}$ 的存储概率分布为 $\{\bar{a}_{i-1}, \bar{b}_i, \dots, \bar{b}_j, \bar{a}_j\}$, 其中, a_h, b_k 分别是下面的条件概率:

$$\bar{b}_k = b_k / w_{ij}, \quad i \leq k \leq j; \quad \bar{a}_h = a_h / w_{ij}, \quad i-1 \leq h \leq j$$

- 设 T_{ij} 是有序集 $\{x_i, \dots, x_j\}$ 关于存储概率分布为 $\{a_{i-1}, b_i, \dots, b_j, a_j\}$ 的一棵最优二叉搜索树, 其平均路长为 p_{ij} , T_{ij} 的根顶点存储的元素 x_m , 其左子树 T_l 和右子树 T_r 的平均路长分别为 p_l 和 p_r 。由于 T_l 和 T_r 中顶点深度是它们在 T_{ij} 中的深度减1, 所以得到

左子树的
搜索概率

右子树的
搜索概率

$$w_{ij} p_{ij} = w_{i, m-1} (p_l + 1) + w_{m, m} + w_{m+1, j} (p_r + 1)$$

$$= w_{ij} + w_{i, m-1} p_l + w_{m+1, j} p_r \quad i \leq j$$

4 最优子结构性质

由于 T_l 是有序集 $\{x_i, \dots, x_{m-1}\}$ 的一棵二叉搜索树，故 $p_l \geq p_{i,m-1}$ 。若 $p_l > p_{i,m-1}$ ，则 $T_{i,m-1}$ 替换 T_l 可得到平均路长比 T_{ij} 更小的二叉搜索树。这与 T_{ij} 是最优二叉搜索树矛盾。同理可证， T_r 也是一棵最优二叉搜索树。因此，最优二叉搜索树问题具有最优子结构性质。

构造最优二叉搜索树时，可以选择先构造其左右子树，使其左右子树最优，然后构造整棵树。

5 递归计算最优值

- 最优二叉搜索树 T_{ij} 的平均路长为 p_{ij} ，则所求的最优值为 $p_{1,n}$ 。由二叉树的花费公式

$$\begin{aligned}w_{ij}p_{ij} &= w_{i,k-1}(p_{i,k-1} + 1) + w_{k,k} + w_{k+1,j}(p_{k+1,j} + 1) \\ &= w_{ij} + w_{i,k-1}p_{i,k-1} + w_{k+1,j}p_{k+1,j} \quad i \leq j\end{aligned}$$

- 根据最优二叉搜索树问题的最优子结构性质可建立计算 p_{ij} 的递归式如下

$$w_{ij}p_{ij} = w_{ij} + \min_{i \leq k \leq j} \{w_{i,k-1}p_{i,k-1} + w_{k+1,j}p_{k+1,j}\}, \quad i \leq j$$

初始时 $p_{i,i-1} = 0, \quad 1 \leq i \leq n$

5 递归计算最优值

■记 $w_{ij}p_{ij}$ 为 $m(i, j)$

$m(1, n) = w_{1,n}p_{1,n} = p_{1,n}$ 为所求最优值。

记 $w_{ij}p_{ij}$ 为 $m(i, j)$ ，得关于 $m(i, j)$ 的递推公式

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$

$$m(i, i-1) = 0, \quad i = 1, 2, \dots, n$$

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$

$$m(i, i-1) = 0, \quad i = 1, 2, \dots, n$$

根据该公式，计算树**T[i][j]**的花费只用到了**T[i][k-1]**，**T[k+1][j]**，可得到具体求解过程如下：

- 1) 构造只有**1**个内部结点的最优二叉搜索树
T[1][1], T[2][2]...，可以求得**m[i][i]** 同时可以用一个数组存做根结点元素为：
s[1][1]=1, s[2][2]=2...s[n][n]=n
- 2) 构造具有**2**个内部结点的最优二叉搜索树

5 递归计算最优值

例

给出标识符集 $\{1, 2, 3\} = \{\text{do}, \text{if}, \text{stop}\}$ 存取概率

若 $P_1=0.5, \quad P_2=0.1, \quad P_3=0.05,$

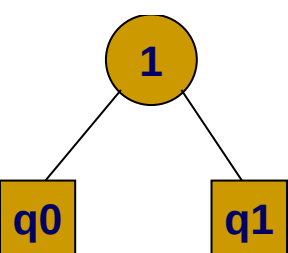
$q_0=0.15, \quad q_1=0.1, \quad q_2=0.05, \quad q_3=0.05$

构造一棵最优二叉搜索树

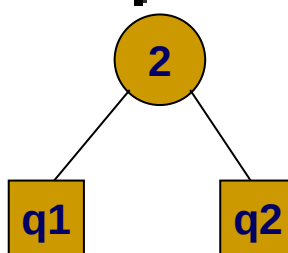
$q_0=0.15, P_1=0.5, q_1=0.1, P_2=0.1, q_2=0.05, P_3=0.05, q_3=0.05$

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$

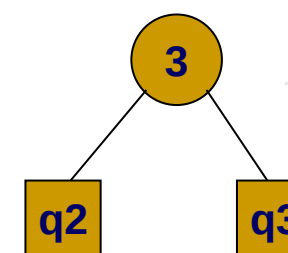
$$m(i, i-1) = 0, \quad i = 1, 2, \dots, n$$



$T[1][1]$
 $w[1][1]=0.75$
 $m[1][1]=0.75$



$T[2][2]$
 $w[2][2]=0.25$
 $m[2][2]=0.25$



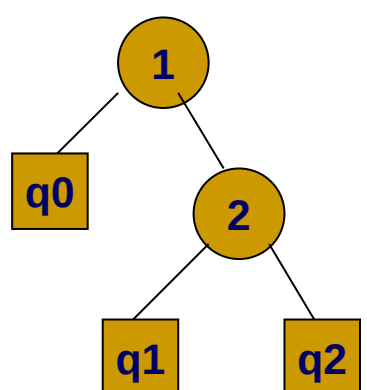
$T[3][3]$
 $w[3][3]=0.15$
 $m[3][3]=0.15$



$T[1][0]$
 $w[1][0]=0.15$
 $m[1][0]=0$

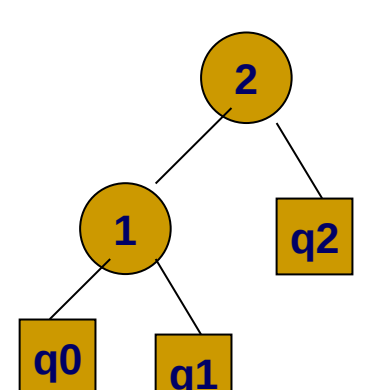


$T[2][1]$
 $w[2][1]=0.1$
 $m[2][1]=0$



$w[1][2]=0.9$
 $m[1][2]=0.9+$
 $m[1][0]+m[2][2]$
 $=1.15$

$T[1][2]$



$w[1][2]=0.9$
 $m[1][2]=0.9+$
 $m[1][1]+m[3][2]$
 $=1.65$



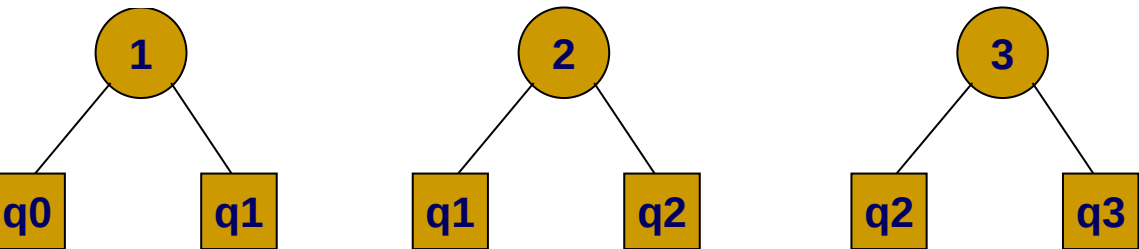
$T[3][2]$
 $w[3][2]=0.05$
 $m[3][2]=0$



$T[4][3]$
 $w[4][3]=0.05$
 $m[4][3]=0$

q0=0.15, ~~P1=0.5~~, q1=0.1, ~~P2=0.1~~, q2=0.05, ~~P3=0.05~~, q3=0.05

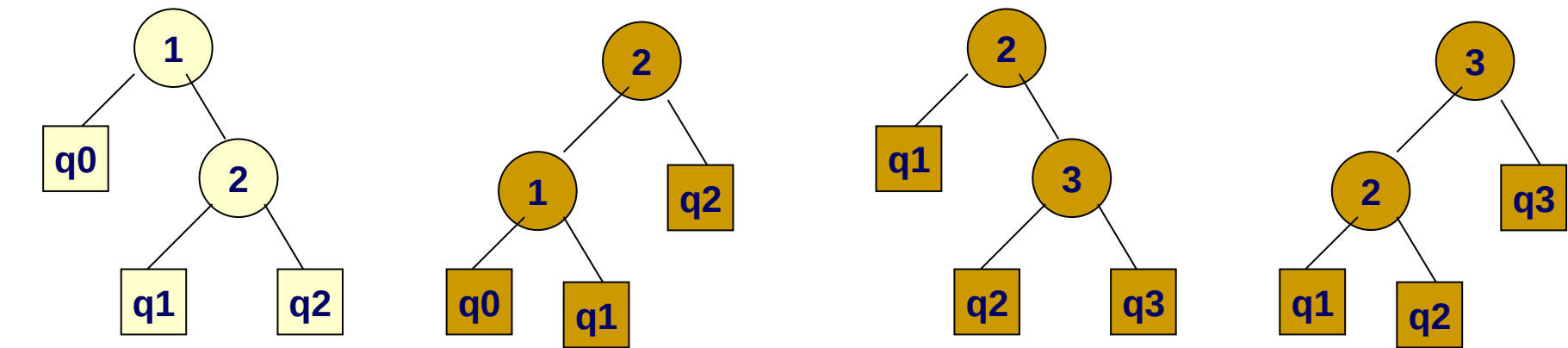
$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k - 1) + m(k + 1, j)\}, \quad i \leq j$$



T[1][1]
w[1][1]=0.75
m[1][1]=0.75

T[2][2]
w[2][2]=0.25
m[2][2]=0.25

T[3][3]
w[3][3]=0.15
m[3][3]=0.15



w[1][2]=0.9
m[1][2]=0.9+
m[1][0]+m[2][2]
=1.15

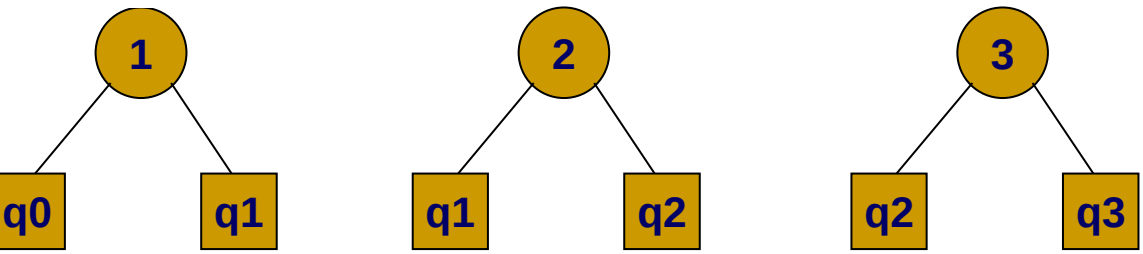
T[1][2]
w[1][2]=0.9
m[1][2]=0.9+
m[1][1]+m[3][2]
=1.65

m[2][3]=0.5

T[2][3]
w[2][3]=0.35
m[2][3]=0.6

q0=0.15, ~~P1=0.5~~, q1=0.1, ~~P2=0.1~~, q2=0.05, ~~P3=0.05~~, q3=0.05

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k - 1) + m(k + 1, j)\}, \quad i \leq j$$



$T[1][1]$

$w[1][1]=0.75$

$m[1][1]=0.75$

$T[2][2]$

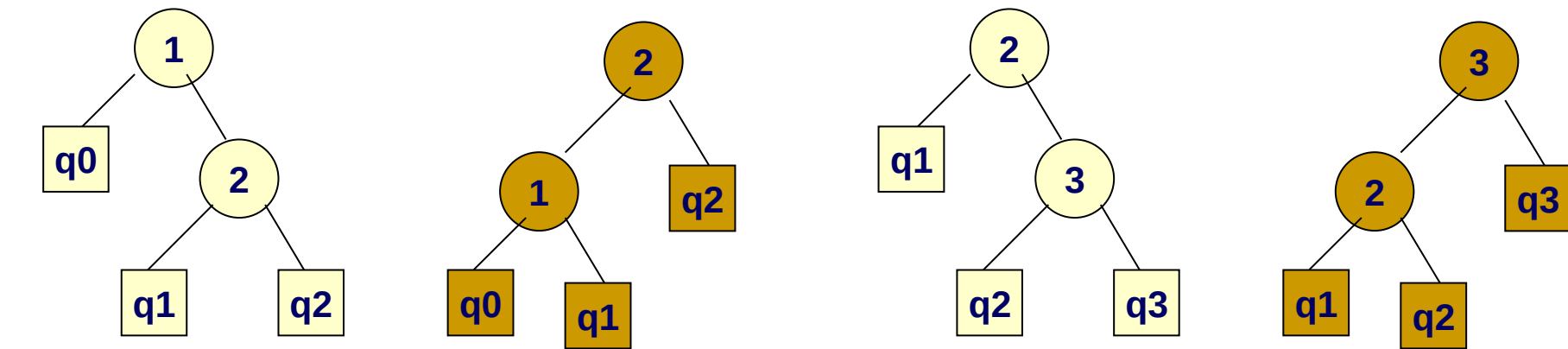
$w[2][2]=0.25$

$m[2][2]=0.25$

$T[3][3]$

$w[3][3]=0.15$

$m[3][3]=0.15$



$w[1][2]=0.9$

$m[1][2]=0.9+$

$m[1][0]+m[2][2]$

$=1.15$

$T[1][2]$

$w[1][2]=0.9$

$m[1][2]=0.9+$

$m[1][1]+m[3][2]$

$=1.65$

$T[2][3]$

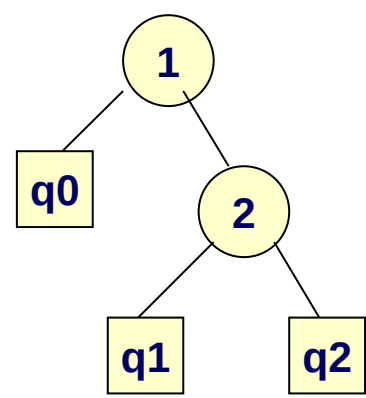
$w[2][3]=0.35$

$m[2][3]=0.5$

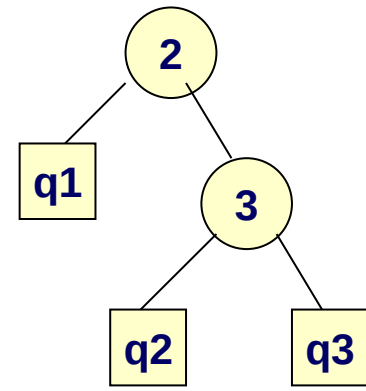
$m[2][3]=0.6$

~~q0=0.15, P1=0.5, q1=0.1, P2=0.1, q2=0.05, P3=0.05, q3=0.05~~

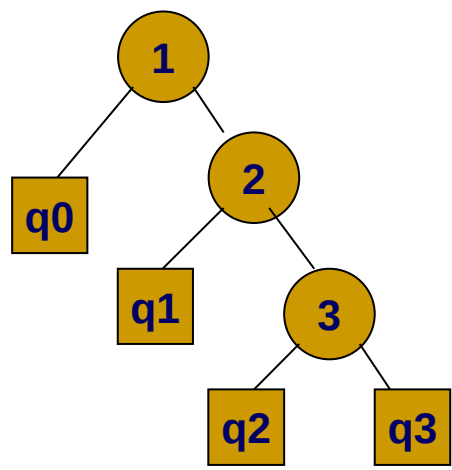
$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$



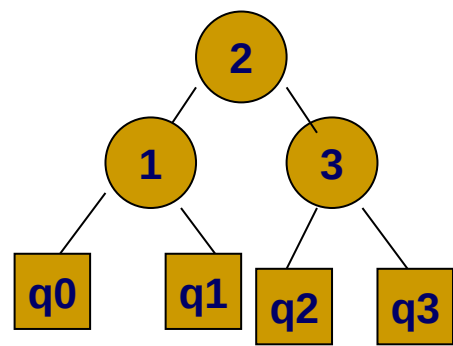
T[1][2]
m[1][2]=1.15



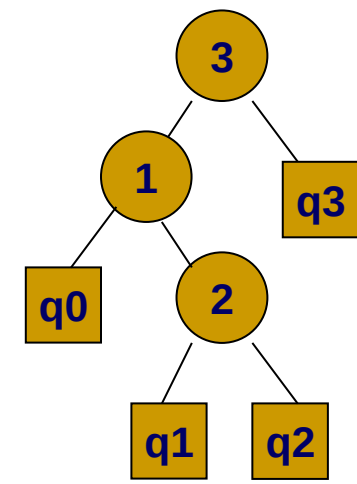
T[2][3]
m[2][3]=0.5



m[1][3]=1.5



m[1][3]=1.9

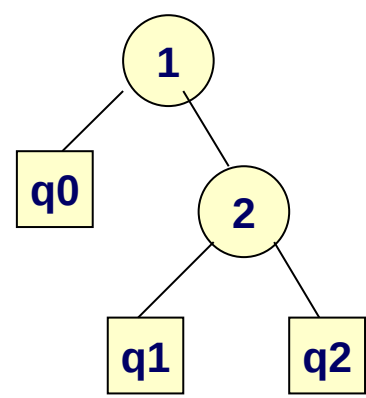


m[1][3]=2.15

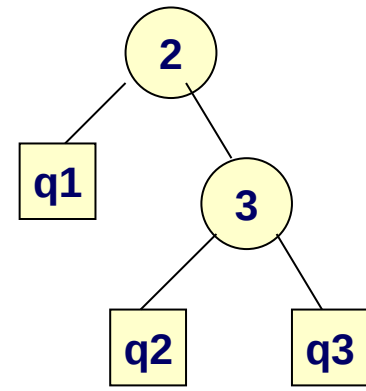
T[1][3]
W[1][3]=1

$q_0=0.15$, $P_1=0.5$, $q_1=0.1$, $P_2=0.1$, $q_2=0.05$, $P_3=0.05$, $q_3=0.05$

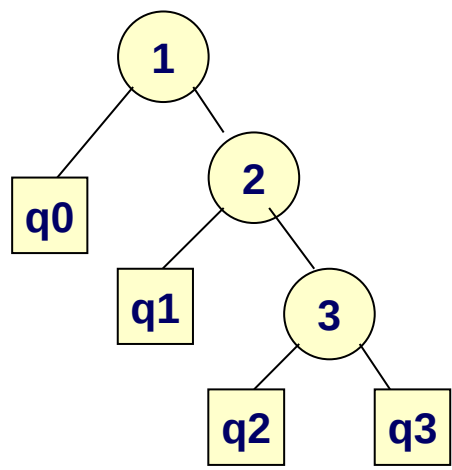
$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$



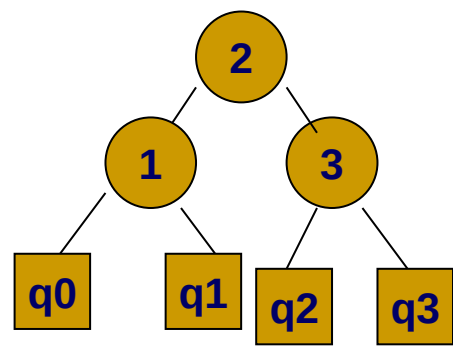
$T[1][2]$
 $m[1][2]=1.15$



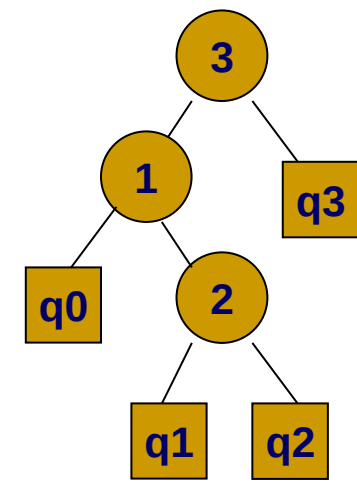
$T[2][3]$
 $m[2][3]=0.5$



$m[1][3]=1.5$



$m[1][3]=1.9$



$m[1][3]=2.15$

$T[1][3]$
 $w[1][3]=1$

$q_0=0.15$, **$P_1=0.5$** , $q_1=0.1$, **$P_2=0.1$** , $q_2=0.05$, **$P_3=0.05$** , $q_3=0.05$

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}, \quad i \leq j$$

	0	1	2	3		0	1	2	3
1	0.15	0.75	0.9	1	1	0	0.75	1.15	1.5
2		0.1	0.25	0.35	2		0	0.25	0.5
3			0.05	0.15	3			0	0.15
4				0.05	4				0

	W(i, j)					m(i, j)			
	0	1	2	3		0	1	2	3
1	0	1	1	1					
2		0	2	2					
3			0	3					
4				0					

	0	1	2	3	
1	0	1	1	1	
2		0	2	2	
3			0	3	
4				0	

s(i, j)

5 递归计算最优值

具体求解过程

- 1) 递归出口，没有内部节点时，构造 $T[1][0]$
 $T[2][1], T[3][2], \dots, T[n+1][n]$
- 2) 构造只有1个内部结点的最优二叉搜索树 $T[1][1], T[2][2], \dots, T[n][n]$ ，可以求得 $m[i][i]$ 同时可以用一个数组存做根结点元素为： $s[1][1]=1, s[2][2]=2 \dots s[n][n]=n$
- 3) 构造具有2个、3个、……、 n 个内部结点的最优二叉搜索树
 r (起止下标的差)
 $0 \quad T[1][1], T[2][2], \dots, T[n][n],$
 $1 \quad T[1][2], T[2][3], \dots, T[n-1][n],$
 $2 \quad T[1][3], T[2][4], \dots, T[n-2][n],$
 $\vdots$
 $r \quad T[1][r+1], T[2][r+2], \dots, T[i][i+r], \dots, T[n-r][n]$
 $\vdots$
 $n-1 \quad T[1][n]$

```

void OBST ( int *a, int *b,int n, int **m, int **s, int **w)
{
    for( int i=0; i<=n; i++)
    {
        w[i+1][i]=a[i];
        m[i+1][i]=0;
    }//初始化，构造没有内部节点时的情况
    for(int r=0; r<n; r++)
        for(int i =1; i<=n-r; i++)
        {
            int j= i+r;
            构造T[i][j]，填写w[i][j],m[i][j],s[i][j]
        }
}

```

5 递归计算最优值

构造 $T[i][j]$

$T[i][j]$ 表示用第 i 到第 j 个内部节点构造的树，做根的结点可以是第 $i, i+1, \dots, j$ 中任意一个。

1) 首选 i 作为根，其左子树空，右子树为结点 $i+1, i+2, \dots, j$ 构成即 $T[i+1][j]$ 。

$$m[i][j] = w[i][j] + 0 + m[i+1][j]$$

$$s[i][j] = i$$

2) 不选 i 做根，设 k 为其根，则 $k = i+1, \dots, j$ ，左子树为结点 $i, i+1, \dots, k-1$ ，右子树为 $k+1, k+2, \dots, j$

$$t = w[i][j] + m[i][k-1] + m[k+1][j]$$

$$\text{if}(t < m[i][j])$$

$$\{ m[i][j] = t;$$

$$s[i][j] = k;$$

$$\}$$

3) $k = k+1$ ，跳回2

5 递归计算最优值

取第k个结点作根

```
void OptimalBinarySearchTree( int *a, int
*b,int n, int **m, int **s, int **w)
```

```
{
  for( int i=0; i<=n; i++){
```

```
    w[i+1][i]=a[i];
    m[i+1][i]=0;
```

初始化
对角线赋值

```
  }
  for(int r=0; r<n; r++){
```

i为起始元素下标

```
    for(int i =1; i<= n - r; i++){
```

```
      int j = i + r;
```

j为终止元素下标

```
      w[i][j]=w[i][j-1]+a[j]+b[j];
```

```
      m[i][j]=m[i+1][j];
```

```
      s[i][j] = i;
```

如第i个结点作根的
值

加第j个结点后,
权值w改变

```
        for(int k = i+1; k<=j; k++){
          int t=m[i][k-1]+m[k+1][j];
          if (t<m[i][j]){
            m[i][j]=t;
            s[i][j]=k;}
        }
      }
    }
```

6 构造最优解

算法 `OptimalBinarySearchTree` 中用 $s[i][j]$ 保存最优子树的根顶点中元素。当 $s[1][n] = k$ 时, x_k 为所求二叉搜索树的根顶点元素。其左子树为 $T(1, k-1)$, 因此 $i = s[1][k-1]$ 即表示 $T(1, k-1)$ 的根顶点为 x_i 。依次类推, 容易由 s 记录的信息在 $O(n)$ 时间内构造出所求的最优二叉搜索树。

7 计算复杂性

算法中用到三个数组 m 、 s 、 w ，故所需的空间为 $O(n^2)$ 。算法的主要计算量在于计算 $w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}$ 。对于固定的差 $r=j-i$ ，需要计算时间 $O(j-i+1) = O(r+1)$ 。因此算法所耗费的总时间为 $\sum_{r=0}^{n-1} \sum_{i=1}^{n-r} O(r+1) = O(n^3)$ 。